

Phase 0a + étape 1a : ce que la mesure a corrigé en route

Force Tracker — **ft-v1193**, en ligne et vérifiée. Quatrième document de la série : DOUANE (les écritures) -> ARCHI (le module) -> PLAN (l'ordre) -> **celui-ci, l'exécution**. Michel : « exécute la phase 0 puis l'étape 1a, avec les témoins et les critères écrits dans ce plan », puis « pour savedFoods, ne corrige rien. Documente le bug et garde-le ouvert. »

L'INTÉRÊT DE CE DOCUMENT N'EST PAS LE CORRECTIF — C'EST CE QUI L'A CORRIGÉ

Le code livré tient en peu de lignes. **Trois choses l'ont modifié pendant l'exécution, et aucune ne venait d'une relecture :**

1. Le plan prescrivait un design — « *un objet qui porte les deux faces* » — qui **rendait son propre critère de réussite invérifiable**. La mesure l'a dit avant la première ligne.
2. La passe complète a trouvé une **vraie régression** du correctif, sur un chemin sans rapport apparent.
3. Ma première réparation de cette régression était **fausse**, et sa forme d'erreur est la même que celle que l'étape était censée supprimer.

1. Ce qui est en ligne, vérifié dans le dépôt à la génération

ce qui était demandé	état mesuré maintenant
Phase 0a — corriger <code>_bcPaquetG</code>	en ligne · garderPaquet présent à 4 endroits
... avec le témoin prévu	14 témoins permanents, bloc CCXC
... avec la mutation prévue	8 mutations , toutes mordent (détail plus bas)
savedFoods — ne rien corriger	0 occurrence dans <code>_fusionListe</code> — intact
... le documenter et le garder ouvert	<code>docs/JOURNAL-DE-TEST.md</code> , état ouvert
Étape 1a — le constructeur unique	0 constructeur à la main · 0 traduction restante · 9 occurrences de <code>_ref100</code> · 5 de <code>_per100De</code>

Le code livré, extrait du dépôt. Il remplace **8** constructions écrites à la main et **4** lignes de traduction identiques au caractère près.

```
function _ref100(nom, kcal, prot, carbs, fat, opts){
  const n = (opts && opts.normaliser===false) ? (x=>+x||0) : _per100d1;
  return { name:String(nom===null?'':nom).slice(0, (opts && opts.maxNom) || 60),
    kcal100:n(kcal), prot100:n(prot), carbs100:n(carbs), fat100:n(fat) };
}
function _per100De(r){
  return r ? {kcal:r.kcal100, prot:r.prot100, carbs:r.carbs100, fat:r.fat100} : null;
}
```

C'EST UNE EXTRACTION, PAS UNE UNIFORMISATION — et c'est tout le contrat

2 portes sur 8 n'arrondissent pas : les *reprises* (« Mes aliments » et la recherche dans le journal), qui recopient un pour-100 g **déjà stocké**. Leur faire traverser `_per100d1` changerait une **valeur enregistrée** : ce serait une **DÉCISION**, pas un rangement. D'où `{normaliser:false}`.

On extrait ce qui existe, on ne redresse rien au passage. Le même raisonnement vaut pour `maxNom` : les portes ne coupent pas toutes le nom pareil (60 partout, **80** pour l'étiquette recopiée à la main). Cet écart est **transporté tel quel**, pas harmonisé — sinon l'instantané bougerait, et on ne saurait plus rien.

2. Le critère était binaire, et il est atteint

Le plan exigeait : « aucune valeur attendue modifiée ». **Une passe verte ne prouve pas ça** — elle prouve que ce que les témoins **regardent** n'a pas bougé. D'où un instantané dédié (tools/instantane_ref100.js) qui sérialise la sortie **brute** des 8 portes, avant et après.

/!\ Cette empreinte est une **sortie datée**, pas un chiffre recalculé à la génération de ce PDF : l'instantané demande un navigateur (~40 s). La commande exacte est donnée pour qu'on puisse la rejouer.

```
node tools/instantane_ref100.js > /tmp/avant.json      # avant de toucher au code
node tools/instantane_ref100.js > /tmp/apres.json      # apres
diff /tmp/avant.json /tmp/apres.json                  # -> vide

resultat mesure le 11/09/2026 : IDENTIQUE octet pour octet
2920 octets / 2920 octets      sha256 (16 premiers) : a8065e734063ecd5
```

LE PLAN PRESCRIVAIT UN DESIGN QUI RENDAIT SON PROPRE CRITÈRE INVÉRIFIABLE

Le plan disait : « _ref100() rend un objet qui porte les deux faces » — kcal100... pour les lecteurs actuels **et** per100 pour la provenance. Élégant sur le papier.

NON Mais poser per100 DANS _bcNutr aurait ajouté un champ à l'objet que l'instantané sérialise. Le critère (*identique octet pour octet*) aurait été violé **par construction** : impossible de distinguer « rien n'a changé » de « tout a changé un peu ».

Deux petites fonctions au lieu d'une maligne. Un critère qu'on est obligé d'assouplir pour faire passer son propre code n'est plus un critère.

3. La passe a trouvé une vraie régression — sur un chemin sans rapport

Deux témoins de **ft-v1174** (bloc CCLXXII (8) et (9)) sont devenus rouges : une ratatouille **trouvée mais sans valeurs** perdait son « 250 g » en partant au calibrage. Or le correctif de la phase 0a ne parle **ni de calibrage ni de ratatouille**.

```
avant : _afOublierAliment ne nettoyait PAS _bcPaquetG
        -> _bcPaquetTxt traversait le chemin du calibrage
        -> parce que PERSONNE ne la nettoyait, pas parce qu'on l'avait voulu

apres : le menage est fait -> la variable est effacee -> le 250 g disparaît
```

NOUVELLE FAMILLE — BUGS.md §62

Une protection qui ne tenait que par l'absence de ménage.

Le jour où l'on range enfin, on découvre ce qui ne tenait que par le désordre.

Ce qui l'attrape : la passe complète, et elle seule. Ni la relecture, ni le contrôle négatif du correctif (qui mordait parfaitement sur **ses** témoins), ni l'instantané avant/après — identique octet pour octet, parce que le chemin cassé **n'y figurait pas**.

Le réflexe : après un correctif qui **nettoie** un état partagé, chercher **qui comptait sur le fait qu'il ne soit PAS nettoyé**.

4. Et ma première réparation était fausse — sa forme vaut plus que le correctif

J'ai recopié le patron « on prend, on oublie, on repose » chez _lookupBarcode, puis chez _calAppliquer. **Le témoin est resté rouge** : il existe une **TROISIÈME** porte sur ce chemin, _bcSansValeurs.

R8, LA PORTE JUMELLE — À L'INTÉRIEUR DU CORRECTIF CENSÉ FERMER UNE FUITE

Un patron qu'on recopie à chaque porte EST la duplication qu'on prétend supprimer.

Le bon geste : un paramètre nommé sur le propriétaire unique — _afOublierAliment({garderPaquet:true}) — qui dit **une seule fois** la distinction entre « j'oublie l'aliment » et « je remets l'écran à plat pour le **MÊME** aliment ». **2 appelants sur 15** sont dans le second cas.

5. Le contrôle négatif : 8 mutations, toutes mordent

#	ce qu'on casse exprès	ce que le banc répond
(1)	`_ref100` rend `null`	RUNNER MORT - mord au maximum
(2)	`_per100d1` retiré (tout devient brut)	1 rouge
(3)	`_per100De` rend un objet VIDE	1 rouge
(4)	le poids du paquet n'est plus effacé	3 rouges
(5)	la variable est propre, l'ÉCRAN n'est pas repeint	2 rouges
(6)	`garderPaquet` ignoré (efface pour tous)	2 rouges , exactement la ratatouille
(7)	`{normaliser:false}` ignoré	2 rouges
(8)	`maxNom` ignoré (tout coupé à 60)	1 rouge

ET LE HARNAIS DE MUTATION M'A MENTI — MÊME FAMILLE QUE §61

La mutation (1) affichait « **0 rouges** ». Je l'ai lue comme « *elle ne mord pas* » — donc comme la preuve que le code muté était **décoratif**.

En regardant la sortie **complète** : le runner **plantait** (`TypeError` sur `null`), donc il n'affichait **rien**, et mon `grep -c` comptait 0 sur une sortie **vide**.

Un compteur de rouges doit d'abord vérifier que le runner a FINI. « 0 rouge » et « n'a pas tourné » produisent le même chiffre.

!! Et deux autres formes de la même famille le même jour : un témoin qui comptait **le commentaire citant le motif supprimé** ; et `pgrep -f "node tests/..."` qui **matche son propre shell**, d'où une attente infinie sur une passe déjà terminée.

* Et un témoin plus ancien a fait son travail contre moi

Le contrôle « *les constructions de `_bcNutr` sont bien trouvées* » a rougi — parce que son voisin (« *aucune construction ne ré-arrondit* ») était passé **VERT sur une liste VIDE**. Sans lui, l'étape 1a aurait transformé une vraie garantie en vert décoratif, **en silence**. Réécrit sur le propriétaire unique : **la garantie ne s'affaiblit pas, elle se déplace**.

6. Deux points mesurés et volontairement NON corrigés

(1) S. `savedFoods` se perd entre deux onglets — décision de Michel : « **ne corrige rien pour l'instant, je ne veux pas d'une rustine qui ressuscite des favoris supprimés** ».

```
onglet A écrit « Pain »           -> disque : ["Pain"]
onglet B (liste perimee) écrit « Fromage » -> disque : ["Fromage"]  <- Pain perdu
TEMOIN foodLog, exactement le meme scenario -> ["Fromage","Pain"]    <- garde
```

NON Et l'ajouter à `_fusionListe` serait **FAUX** : la fusion fait une **union par clé**. Les 5 listes qu'elle protège sont des **journaux qui ne font qu'AJOUTER** ; les favoris se **SUPPRIMENT**. Une fusion par union sur une liste où l'on retire n'est pas une protection, c'est une **résurrection**. Il faut un horodatage ou une pierre tombale — **décision produit, pas rustine**.

(2) Le scan rend 48, 3, la recherche par nom rend 48 — sur la même fiche.

Trouvé **par l'instantané**, alors que personne ne le cherchait : `_afSuggKcal100` réécrit la formule de `_lookupBarcode` (`energy-kcal_100g || energy_100g/4.184`) avec `Math.round` au lieu de `_per100d1`. 9e occurrence de la famille `BUGS.md §59`. **NON Non corrigé exprès** : corriger changerait une **valeur enregistrée**, donc ce ne serait plus une extraction. Et c'est cette promesse qui rend l'instantané lisible.

ÉTAT DES TESTS ET DU DÉPLOIEMENT

Parcours 3549/3549 sur l'arbre final (+14, bloc CCXC) · calculs 339/339 · muscles 241/241 · croisés 50/50 · dates 9/9 · données classées 0 trou.

OK Déploiement vérifié : run #1087, job success, 5 étapes vertes, terminé à **20:49:12 UTC**.

/!\ Une limite dite : l'API a affiché le **run** en `in_progress` alors que son **job** était terminé depuis plusieurs minutes — un état périmé, pas un blocage. *Ce n'est pas UNE requête qui est fiable, c'est d'en croiser DEUX.* Et le proxy de ce conteneur refuse `github.io` : l'app affichant **ft-v1193** reste à confirmer sur iPhone.

CE QUE CE DOCUMENT N'EST PAS

Les compteurs de la page 1 sont **recalculés à chaque génération** depuis le dépôt, et le générateur **refuse de produire** si l'un d'eux redevient faux — notamment si savedFoods était fusionné, puisque Michel a demandé de le laisser **ouvert**.

/!\ En revanche, l'empreinte de l'instantané et le décompte des mutations ne sont pas recalculés ici (navigateur, 8 passes de banc). Le document cite des sorties **datées** et nomme les commandes pour les rejouer. *Un chiffre mesuré qu'on ne peut pas rejouer est un chiffre qu'il faut croire.*

SUITE Suite du plan : étapes 1b, 2 et 3 (extractions, même critère d'instantané), puis le hub (phase 2), puis la douane en mode observateur (phase 3).